Geometry-Based Target Tracking

with Gated SAM 2 Refinement

Implementation brief for SmartData

Derek Huang November 5, 2026

1. Objective and system concept

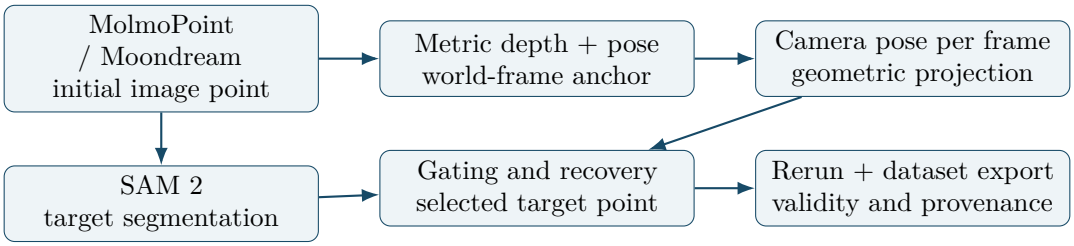
Extend the existing MolmoPoint/Moondream and SAM 2 branch with a persistent 3D target. Camera motion predicts where this target should appear in each image; segmentation supplies an independent visual check and limited refinement.

Proposal status. This document specifies an extension, not functionality already implemented. The reviewed repository is SmartData commit `053227d`. The initial milestone uses recorded iPhone poses; robot wrist-camera deployment follows after geometry validation.

Two complementary estimates

Estimate	Source	Typical failure
Geometry	Reproject a 3D anchor through camera pose	Calibration error, pose drift, moving object
Visual	SAM 2 target mask and a defined point within it	Occlusion, mask drift, wrong instance

Proposed data flow



Interpretation of the peer's suggestion

The supplied comparison table accepts SAM 2 refinement when its point falls *within* 30 pixels of the geometric prediction. Large disagreement is evidence of a possible failure in either branch. It should trigger verification or re-detection rather than unconditional replacement by SAM 2.

Scope of the first version

Use one stationary target during the approach phase, process recorded trials offline, and visualize both estimates before changing policy inputs. Preserve hindsight contact labels as a separate evaluation stream.

2. Calibration and required capture data

Inputs that must be available

Input	Requirement
Camera intrinsics	Matrix K, lens distortion, image resolution, and camera identity
Metric depth	Depth in declared units, depth convention, invalid-value mask, and timestamps
RGB–depth alignment	Registration into the RGB image or a calibrated mapping between sensors
Camera trajectory	Timestamped poses with explicit axes and transform direction
Robot wrist calibration	Camera-to-gripper transform, joint states, and robot kinematic model

Current blocker. The repository's depth MP4 is a visualization, not a metric depth measurement. Calibration and a 2D point alone define a ray; depth or another geometric constraint is needed to locate a 3D target.

Coordinate convention

Use T_{AB} to map points from frame B into frame A. Let W be the world or robot base, G the gripper, and C the optical camera frame. A bar denotes homogeneous coordinates. Use meters for translation and a documented quaternion ordering.

For the iPhone prototype, construct T_{WC_t} from the recorded pose after verifying ARKit pose direction and converting its camera axes to the optical convention. Do not assume an ARKit camera frame is identical to the OpenCV optical frame.

For a robot wrist camera:

$$T_{WC_t} = T_{WG_t}(q_t)\, T_{GC}, \tag{1}$$

where q_t contains joint angles, forward kinematics provides T_{WG_t}, and hand–eye calibration estimates the fixed camera-to-gripper transform T_{GC}.

Calibration procedure

1. Estimate intrinsics using a checkerboard or ChArUco board at varied image positions and orientations. Keep the capture settings consistent with deployment.
2. For a robot camera, collect synchronized gripper poses and calibration-board observations over varied wrist rotations and translations.
3. Estimate T_{GC} with hand–eye calibration. Use OpenCV functions `calibrateCamera` for intrinsics and `calibrateHandEye` for the fixed transform [2].
4. Validate on held-out observations and a stationary target viewed from multiple camera poses. Report reprojection error and 3D anchor consistency.

Timing and resolution

Interpolate translation to image timestamps and use quaternion SLERP for orientation. Verify clock offsets and RGB–depth timing. Carry pixel coordinates in native RGB resolution; convert explicitly at SAM input, preview, and policy boundaries. Intrinsics must follow every resize or crop.

3. Initialize and project the 3D anchor

Initial grounding and depth selection

Run the selected grounding model at an explicit seed frame t_0. Preserve the object ID, text prompt, frame index, and native image coordinates (u_0, v_0). Seed SAM 2 at that same frame.

Reject non-finite or out-of-image points. Estimate depth from valid nearby pixels inside the target mask, avoiding discontinuities and background contamination. If no trustworthy depth is available, report a failed initialization rather than inventing a 3D point.

For an undistorted image, let

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}. \tag{2}$$

If z_0 is optical-axis depth, deprojection gives

$$p_{C_0} = z_0 K^{-1} \begin{bmatrix} u_0 \\ v_0 \\ 1 \end{bmatrix} = \begin{bmatrix} (u_0 - c_x) z_0 / f_x \\ (v_0 - c_y) z_0 / f_y \\ z_0 \end{bmatrix}. \tag{3}$$

If depth is radial range, normalize the ray before multiplying by range. For raw distorted pixels, undistort the point before applying this equation [2].

Store the persistent target in the world frame:

$$\bar{p}_W = T_{WC_0} \bar{p}_{C_0}. \tag{4}$$

Per-frame geometry prediction

For a stationary target:

$$\bar{p}_{C_t} = T_{WC_t}^{-1} \bar{p}_W, \tag{5}$$

$$u_t^{\text{kin}} = \begin{bmatrix} f_x X_t / Z_t + c_x \\ f_y Y_t / Z_t + c_y \end{bmatrix}, \qquad p_{C_t} = (X_t, Y_t, Z_t). \tag{6}$$

The superscript "kin" matches the supplied comparison table. With iPhone recordings, the pose source is odometry rather than robot kinematics.

Reject $Z_t \leq 0$ and projections outside the image. Use distortion-aware projection when comparing against raw distorted RGB. An in-image projection does not establish visibility: an occluder may be closer than the anchor.

Anchor persistence and object motion

A world-fixed anchor is appropriate before grasping a stationary object. During transport, switch to a gripper-attached anchor only after validating a stable grasp, or re-estimate object motion. Release or a new task phase requires an explicit anchor transition.

Important distinction. Refining a 2D policy point does not update the 3D world anchor. A persistent 3D update needs reliable depth, identity checks, and an explicit acceptance rule.

4. Gated visual refinement and recovery

Compare equivalent target definitions

The existing tracker exports the target mask centroid. That centroid changes under occlusion and viewpoint change, so it is not generally the projection of a fixed physical surface point.

For the first implementation, test whether the geometric projection lies inside the SAM mask. Keep it if it does. If it lies just outside, use a nearby interior mask point as a candidate correction. Large point-to-mask distance indicates disagreement. A centroid-based comparison is suitable only if the chosen target definition makes the two estimates comparable.

Gate and selection rule

When comparable visual and geometric points are available, compute

$$r_t = \left\| u_t^{\text{sam}} - u_t^{\text{kin}} \right\|_2 . \tag{7}$$

Let V_t^{sam} denote a valid mask and visual candidate. A proposed rule is

$$u_t^{\text{out}} = \begin{cases} (1-\alpha)u_t^{\text{kin}} + \alpha u_t^{\text{sam}}, & V_t^{\text{sam}} \text{ and } r_t \le \tau, \\ u_t^{\text{kin}}, & \text{otherwise, if geometry is valid.} \end{cases} \tag{8}$$

If neither branch is reliable, output an invalid target and reason. The blend weight α and threshold τ are validation parameters, not established performance settings.

Condition	Proposed response
Small agreement residual	Accept gated refinement; retain anchor identity
Missing or unstable mask	Keep geometry; flag visual loss
Persistent large residual	Request a fresh grounding observation; retain disagreement logs
Verified re-detection	Reinitialize anchor using valid depth and reseed SAM 2
No reliable estimate	Mark invalid; expose the reason to downstream consumers

Thresholds, identity, and recovery

The reference gate is 30 pixels at an unspecified image scale. In SmartData, 30 pixels at 720×960 becomes 16 pixels at 384×512. State the coordinate system with every threshold and tune it on held-out trials.

Use mask area changes, temporal consistency, depth validity, and object identity to reject suspect observations. Require persistent disagreement rather than reacting to a single noisy frame. Periodic grounding, such as the reference's roughly two-second schedule, is an optional recovery mechanism to benchmark against event-triggered grounding.

SAM 2 exposes frame-specific corrective prompts and mask propagation [3]. Preserve the re-detection frame index and restart or resume tracking from that frame. Do not treat all detections as frame-0 seeds.

5. Repository integration and training interface

Module	Proposed responsibility
`camera_geometry.py`	Shared transforms, projection, deprojection, calibration, and coordinate conversion
`initialize_target_anchor.py`	Grounding result plus depth and pose to world-frame anchor
`project_target_anchor.py`	Timestamp-aligned geometric prediction per frame
`extract_target_track.py`	Preserve IDs and masks; support arbitrary seed frames
`fuse_target_track.py`	Gate candidates; export validity, source, and recovery requests
`build_gripper_rrd.py`	Compare projections, visual points, fused points, and residuals
`main.py`	Schedule stages and manage calibration-aware dependencies
LeRobot / Zarr converters	Export chosen target coordinates and validity alongside current data

Offline stage order

`ground` → `initialize` → `project` → `track` → `fuse` → `visualize/export`

The existing gripper-aperture and hindsight-contact branch remains a separate producer of demonstration labels. Recovery can initially use a second offline pass: identify disagreement intervals, re-ground selected frames, and rerun affected tracking segments. Live deployment requires a timestamped processing loop and measured compute latency.

Recommended saved fields

Record	Fields
Persistent anchor	Object ID, prompt, seed timestamp, p_W, calibration ID, depth validity
Per-frame diagnostics	Timestamp, object ID, geometric/visual/output pixels, residual, validity, selected source, recovery reason
Training observation	Target coordinates in policy image space, visibility flag, optional camera-frame 3D target

Policy image coordinates

The current exporters resize RGB directly to a square. A centered crop would change both image content and target coordinates. For crop origin (x_0, y_0) and resize scales (s_x, s_y):

$$u' = s_x(u - x_0), \qquad v' = s_y(v - y_0). \tag{9}$$

Mark targets outside the crop as not visible. Apply the identical transform during training and deployment. Keep label-derived contact anchors separate from online target predictions to avoid future-information leakage.

Controller changes

Include calibration, metric depth, pose, seed frame, and model settings in stage dependency checks. Validate all required outputs rather than checking only a CSV. Rebuild downstream products when any geometric input changes.

6. Validation plan and implementation milestones

A staged development sequence

Stage	Deliverable	Acceptance evidence
1	Calibrated capture and geometry	Held-out reprojection checks; correct axes, units, timestamps, and depth alignment
2	One stationary target	Reprojected target follows camera motion across the recorded trajectory
3	SAM 2 comparison and gating	Annotated error, mask-loss flags, and recorded acceptance/rejection reasons
4	Re-detection and recovery	Correct identity recovered after occlusion or deliberate tracker drift
5	Dataset interface	Pixel/crop consistency and identical training/deployment target semantics
6	Robot wrist deployment	Verified hand–eye transform and measured end-to-end latency

Experiments

Evaluate geometry-only, SAM-only, and gated fusion on identical sequences. Begin with a stationary object and camera motion, then introduce partial occlusion, similar nearby instances, and deliberate object motion. Separate calibration/tuning sequences from evaluation sequences.

Report median and tail pixel error against manually annotated points with the same target definition, valid-track fraction, object switches, recovery delay, and inference latency. Report 3D error only when an independent metric reference exists. Include rejection frequency so a low error cannot be achieved merely by discarding difficult frames.

The supplied green-ring contact labels are hindsight references. SmartData's current contact detector is heuristic, so manually verify any events used for evaluation. A contact midpoint and an object centroid are different targets and should not be scored as if they were identical.

Recommended first milestone. Obtain metric depth and intrinsics, initialize one target, and reproject it through an existing iPhone recording. Display the geometric and SAM estimates in Rerun. Validate this offline result before adding automatic anchor updates or robot control.

References and implementation basis

[1] G. D. Huang. *SmartData*. Reviewed commit `053227d`. https://github.com/GuanlinDerekHuang/SmartData.

[2] OpenCV. *Camera Calibration and 3D Reconstruction*. Intrinsic calibration, hand–eye calibration, and image projection. https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html.

[3] Meta. *SAM 2 video predictor source*. Frame-specific prompts and video propagation. https://github.com/facebookresearch/sam2/blob/main/sam2/sam2_video_predictor.py.